

# 01 — Phase 1 Final Deliverables

## ❖ Phase 1 Name: **Architecture Design & Specification**

## ❖ Duration: 1 week

## ❖ Phase 1 Objective

The objective of Phase 1 is to convert the existing AdaptiveDETR/DINOv3 architecture, inference JSON format, and active learning goals into a clear implementation blueprint.

This phase does not train a final model and does not deploy a production system. Instead, it defines exactly what will be built in Phases 2–4 so that implementation can proceed without ambiguity.

## ❖ Current System Facts Used in This Design

### From the supplied architecture markdown:

- The current system has a global backbone/head classifier.
- The global classifier predicts one of 8 domains/heads.
- The current global classifier is effectively a '1536 -> 8' linear classifier.
- The model uses 8 domain-specific detection heads:  
ALL, EC, YM, XSA, ETB, LS, LM, BC
- Bounding box prediction is shared across heads.
- The bounding box format is '[x\_center, y\_center, width, height]'.
- Current inference first performs global classification, then computes logits for all detection heads, then the postprocessor selects/interprets the relevant head.

### From the supplied JSON sample:

- The current response includes keys: 'boxes, proba, labels, count, backbone\_label, hough\_circle, backbone\_label\_string, preferred\_e\_code, preferred\_l\_code'.
- The sample contains 323 predicted boxes.
- The sample includes 8 probability arrays, 8 label arrays, and 8 count arrays.

- The selected/global label in the sample is `XSA`.
- The sample includes OCR-related metadata such as `preferred\_e\_code` and `preferred\_l\_code`.

## ❖ What I Will Deliver in Phase 1

### 1. Architecture Specification

A written system architecture describing:

- Existing model flow.
- Target inference server architecture.
- OCR + global routing logic.
- AdaptiveDETR/DINOv3 inference wrapper.
- Postprocessing structure.
- Active learning module placement.
- Label Studio integration.
- Model versioning and retraining path.
- On-premise deployment boundaries.

### 2. API Specification

A draft OpenAPI/Swagger API spec for:

- `/health`
- `/predict`
- `/batch\_predict`
- `/uncertain\_batch`
- `/feedback`
- `/model/version`

This provides the contract between the AI server and the client's end-user system.

### 3. Data Schema Specification

A normalized schema for:

- Input image request.
- OCR/Petri dish metadata.
- Backbone/domain classification.
- Bounding boxes.
- Per-head predictions.
- Counts.
- Uncertainty values.
- Human-review flags.
- Feedback records.
- Retraining metadata.

#### 4. Active Learning Strategy

A design for:

- Per-box confidence scoring.
- Per-image uncertainty scoring.
- Hybrid sampling using uncertainty + diversity.
- Human review threshold logic.
- Pseudo-labeling safety rules.
- Selection of images for annotation.
- Avoiding redundant annotation requests.

#### 5. Label Studio Workflow

A planned on-prem Label Studio workflow:

- Export uncertain images/tasks.
- Pre-fill model predictions.
- Allow lab analysts to correct outputs.
- Import corrected annotations.
- Convert corrections into retraining-ready format.

## 6. Training and Retraining Strategy

A written plan for:

- Baseline evaluation.
- Model improvement.
- Backbone freezing/fine-tuning decision.
- Head-specific retraining.
- Validation metrics.
- Model promotion criteria.
- Rollback/versioning.

## 7. On-Prem Deployment Plan

A deployment blueprint covering:

- FastAPI server.
- Docker packaging.
- GPU runtime assumptions.
- No image retention policy.
- Logging without storing proprietary image data.
- Local model registry.
- Throughput benchmarking plan.

## 8. Implementation Roadmap

A refined plan for Phases 2~4.

## 02 — System Architecture Specification

### 1. Current Architecture Interpretation

The supplied architecture describes an AdaptiveDETR-style model with:

- A DINOv3/backbone feature extractor.
- A global backbone/domain classifier.
- Multiple domain-specific detection heads.
- A shared bounding box prediction head.
- A postprocessor that selects/interprets outputs using the predicted domain/head.

### 2. Current Head Structure

| Head | Output Shape | Meaning              |
|------|--------------|----------------------|
| ALL  | 256 -> 2     | 1 class + background |
| EC   | 256 -> 3     | 2 class + background |
| YM   | 256 -> 3     | 2 class + background |
| XSA  | 256 -> 3     | 2 class + background |
| ETB  | 256 -> 2     | 1 class + background |
| LS   | 256 -> 2     | 1 class + background |
| LM   | 256 -> 4     | 3 class + background |
| BC   | 256 -> 2     | 1 class + background |
| TC   | 256 -> 2     | 1 class + background |

The shared box head predicts boxes in normalized [x\_center, y\_center, width, height] format.

### 3. Current Runtime Flow

A[Petri dish image] --> B[Preprocessor]

B --> C[Global classify mode]

C --> D[backbone\_logits / head\_id]

B --> E[Inference mode]

E --> F[All detection head logits]

E --> G[Shared predicted boxes]

D --> H[Postprocessor]

F --> H

G --> H

H --> I[Current JSON response]

#### 4. Target Phase 2–4 Architecture

A[Client system] --> |image request| B[FastAPI Inference Server]

B --> C[Image validation + preprocessing]

C --> D[OCR module]

D --> E[Petri dish metadata / routing hints]

C --> F[AdaptiveDETR / DINOv3 model wrapper]

E --> F

F --> G[Raw model outputs]

G --> H[Postprocessor]

H --> I[Normalized inference response]

H --> J[Uncertainty scorer]

J --> K{Review required?}

K --> |No| L[Return final result]

K --> |Yes| M[Label Studio task queue]

M --> N[Lab analyst correction]

N --> O[Feedback import]

O --> P[Retraining dataset builder]

P --> Q[Retraining pipeline]

Q --> R[Validation + model registry]

R --> B

...



## 5. Important Design Decisions

### ➤ Keep Full-Image Context

You want to move away from multiple crops because crops can remove context and slow down inference. The target architecture should preserve full Petri dish image inference where possible.

### ➤ Support OCR-Guided Expert Routing

The OCR system can detect written text and identify the Petri dish type. This should be used as a routing signal for:

- Selecting the most relevant head.
- Reducing unnecessary downstream computation.
- Supporting a future Mixture-of-Experts approach.

### ➤ Avoid Traditional Heavy Ensembles

You don't want 3–5 full models running in parallel because this increases memory and latency. The target uncertainty design should favor:

- Single-model calibrated uncertainty.
- Feature diversity.
- OCR-guided expert selection.
- Lightweight disagreement methods only when practical.

### ➤ Preserve Existing JSON Compatibility

The inference service should preserve the current response format as much as possible, but also add optional fields for uncertainty and review routing.

## 03 — API Specification

Attached API specification YAML file.

# 04 — Data Schema Specification

## 1. Current JSON Response Structure

The uploaded inference JSON sample contains these top-level fields:

```
[
  "boxes",
  "proba",
  "labels",
  "count",
  "backbone_label",
  "hough_circle",
  "backbone_label_string",
  "preferred_e_code",
  "preferred_l_code"
]
```

Observed sample summary:

|                         |                                                 |  |
|-------------------------|-------------------------------------------------|--|
| `boxes`                 | List of normalized bounding boxes.              |  |
| `proba`                 | List of probability arrays, one array per head. |  |
| `labels`                | List of label arrays, one array per head.       |  |
| `count`                 | Count values per head.                          |  |
| `backbone_label`        | Numeric global/domain label.                    |  |
| `backbone_label_string` | Human-readable selected head/domain.            |  |
| `hough_circle`          | Plate/circle metadata from preprocessing.       |  |
| `preferred_e_code`      | OCR-derived E-code.                             |  |
| `preferred_l_code`      | OCR-derived L-code.                             |  |

In the sample:

- Number of boxes: `323`
- Number of heads: `8`
- Selected backbone label: `XSA`
- Preferred E-code: `E10/2026`
- Preferred L-code: `L446502`

## 2. Current Box Format

Boxes are normalized and appear to use:

[x\_center, y\_center, width, height]

Example:

```
[  
  0.7876765131950378,  
  0.18149660527706146,  
  0.009505623951554298,  
  0.008074549958109856  
]
```

## 3. Current Multi-Head Output Pattern

Each head has one probability list and one label list of length equal to the number of predicted boxes.

Recommended fixed head order:

0: ALL

1: EC

2: YM

3: XSA

4: ETB

5: LS

6: LM

7: BC

8: TC

## 4. Proposed Normalized Response Schema

The production API can keep the existing fields for backward compatibility while adding a more explicit `predictions` list.

```
{
  "image_id": "string",
  "model_version": "string",
  "backbone_label": 3,
  "backbone_label_string": "XSA",
  "preferred_e_code": "E10/2026",
  "preferred_l_code": "L446502",
  "hough_circle": [0.4935, 0.37625, 0.44, 0.36667],
  "boxes": [[0.7877, 0.1815, 0.0095, 0.0081]],
  "proba": [],
  "labels": [],
  "count": [],
  "predictions": [
    {
      "box_index": 0,
      "box": [0.7877, 0.1815, 0.0095, 0.0081],
      "active_head_id": 3,
      "active_head_name": "XSA",
      "label": 0,
      "label_name": "XSA_class_0",
      "confidence": 0.9985,
      "uncertainty": 0.0015,
      "requires_review": false
    }
  ],
  "overall_uncertainty": 0.12,
```

```
"review_required": false
}
```

## 5. Review/Active Learning Fields to Add

|                          |         |                                                   |  |
|--------------------------|---------|---------------------------------------------------|--|
| `uncertainty`            | float   | Per-box uncertainty score.                        |  |
| `requires_review`        | boolean | Whether this prediction needs human verification. |  |
| `overall_uncertainty`    | float   | Whole-image active learning score.                |  |
| `review_required`        | boolean | Whether image should go to Label Studio.          |  |
| `active_learning_reason` | string  | low confidence, high entropy, rare class.         |  |
| `model_version`          | string  | Required for tracking feedback against version.   |  |

## 6. Feedback Schema

Corrected annotations from Label Studio should be converted into a consistent feedback format:

```
{
  "image_id": "sample_001",
  "model_version": "adaptive_detr_dinov3_v1.2.0",
  "annotator_id": "analyst_01",
  "corrected_predictions": [
    {
      "box": [0.51, 0.44, 0.02, 0.02],
      "head_name": "XSA",
      "label": 1,
      "label_name": "XSA_positive",
      "correction_type": "label_changed"
    }
  ],
  "review_timestamp": "2026-06-04T10:00:00Z"
}
```

# 05 - Active Learning Uncertainty Design

## 1. Goal

Reduce manual annotation effort by selecting only the most valuable images/colonies for human review.

The system should not ask analysts to review every image. It should send only uncertain, novel, rare, or strategically valuable samples to Label Studio.

## 2. Recommended Strategy

Use hybrid active learning:

Active learning score = uncertainty score + diversity score + rarity/priority score

This avoids a common failure mode where the model sends many similar uncertain images for review.

## 3. Per-Box Uncertainty

For every detected colony/box, compute one or more of:

- Least Confidence

$\text{uncertainty} = 1 - \text{max\_class\_probability}$

- Margin Uncertainty

$\text{uncertainty} = \text{top\_1\_probability} - \text{top\_2\_probability}$

Lower margin means higher uncertainty.

- Entropy

$\text{entropy} = -\sum(p_i * \log(p_i))$

Higher entropy means the model is more confused.

## 4. Per-Image Uncertainty

A Petri dish image can contain hundreds or thousands of colonies. Whole-image uncertainty can be computed using:

```
image_uncertainty = max(per_box_uncertainty)
```

or:

```
image_uncertainty = mean(top_K_uncertain_boxes)
```

Recommended default:

```
image_uncertainty = 0.5 * mean(top_20_box_uncertainties)
                    + 0.3 * max_box_uncertainty
                    + 0.2 * rare_class_priority
```

## 5. Diversity Sampling

To avoid selecting duplicate-looking images, use feature-space diversity:

1. Extract embeddings from the backbone or decoder features.
2. Cluster or compare candidate images.
3. Select a diverse subset from the uncertain pool.

Possible methods:

- CoreSet / k-center greedy.
- Embedding clustering.
- BADGE-style gradient embedding if practical.
- Simple cosine-distance filtering as a Phase 2 baseline.

## 6. Pseudo-Labeling Strategy

High-confidence predictions can be used as pseudo-labels only when strict safety rules are met.

Recommended rules:

- Confidence above a high threshold, e.g. `> 0.95`.
- Prediction passes calibrated confidence threshold.
- Prediction is stable across augmentations if TTA is used.
- Class is not known to be problematic or rare.
- Pseudo-labels are stored separately from human labels.
- Validation must prove pseudo-labels improve performance before promotion.

## **7. Calibration**

Raw confidence is not always reliable. Use calibration to improve uncertainty quality:

- Temperature scaling.
- Expected Calibration Error (ECE).
- Reliability diagrams.
- Per-head threshold tuning.

## **8. Human Review Routing**

A prediction/image should be sent to human review if:

- Confidence is below threshold.
- Entropy is above threshold.
- Margin is below threshold.
- The image belongs to a rare or underperforming class.
- The model detects unusual density or distribution.
- OCR result conflicts with model head prediction.
- The sample is selected by diversity sampling.

## 9. Output of Active Learning Module

The module should output:

```
{  
  "image_id": "sample_001",  
  "overall_uncertainty": 0.78,  
  "rank": 1,  
  "review_required": true,  
  "reasons": [  
    "high_entropy",  
    "low_margin",  
    "rare_class_candidate"  
  ],  
  "selected_boxes": [0, 12, 58]  
}
```

# 06 — Label Studio Workflow

## 1. Goal

Use Label Studio as the on-prem human-in-the-loop review interface.

The analysts should review only selected uncertain samples, not the entire image archive.

## 2. Proposed Flow

flowchart TD

A[Inference server] --> B[Uncertainty scorer]

B --> C[Select images for review]

C --> D[Create Label Studio tasks]

D --> E[Pre-filled model predictions]

E --> F[Lab analyst correction]

F --> G[Export reviewed annotations]

G --> H[Convert to training format]

H --> I[Retraining dataset]

## 3. Label Studio Task Contents

Each review task should include:

- Full Petri dish image.
- Model-predicted bounding boxes.
- Model-predicted class/head.
- Confidence score.
- Uncertainty reason.
- OCR-derived E-code/L-code when available.

- Model version.

#### **4. Analyst Actions**

The analyst should be able to:

- Accept a prediction.
- Correct a class label.
- Move/resize bounding box.
- Delete false positives.
- Add missed colonies.
- Mark image as unusable.
- Add comments for edge cases.

#### **5. Export Format**

Label Studio output should be converted into the internal training format.

The conversion script should preserve:

- `image\_id`
- original model prediction
- corrected label
- corrected box
- correction type
- annotator ID
- timestamp
- model version
- active learning reason

#### **6. Suggested Review Queue Categories**

|                    |                                            |  |
|--------------------|--------------------------------------------|--|
| High uncertainty   | Model confidence is low.                   |  |
| Rare class         | Class has low representation.              |  |
| OCR/model conflict | OCR dish type and predicted head disagree. |  |
| Density challenge  | Very dense colony images.                  |  |
| Validation audit   | Random sample for quality control.         |  |

# 07 — Training and Retraining Strategy

## 1. Goal

Improve the current model while minimizing manual annotation.

The system should support continuous learning from analyst feedback.

## 2. Initial Evaluation

Before changing the model, run a baseline evaluation:

- Per-head precision.
- Per-head recall.
- Per-head F1.
- Bounding box mAP / IoU.
- Count accuracy.
- Calibration error.
- Latency.
- Failure modes by Petri dish type.

## 3. Model Improvement Options

### Option A — Keep DINOv3/AdaptiveDETR and Improve Training

This is the lowest-risk path.

Possible improvements:

- Better calibration.
- Better loss weighting for rare heads/classes.
- Improved query count tuning.
- Better data augmentation.

- Head-specific threshold tuning.
- Hard negative mining.
- Active learning feedback loop.

#### Option B — Add OCR-Guided Mixture of Experts

Use OCR to route the image to the relevant expert/head.

Benefits:

- Lower unnecessary compute.
- Better head specialization.
- Lower memory impact than full ensembles.

#### Option C — Evaluate Alternative Detector Head

If benchmarks justify it, compare the current AdaptiveDETR design against an improved DETR-style head.

This should be done as a controlled experiment, not as a blind replacement.

## **4. Retraining Loop**

flowchart TD

A[New reviewed annotations] --> B[Dataset builder]

B --> C[Train candidate model]

C --> D[Validate on holdout set]

D --> E{Meets promotion criteria?}

E --> |Yes| F[Register model version]

E --> |No| G[Keep current production model]

F --> H[Deploy on-prem inference server]

## **5. Promotion Criteria**

A new model version should be promoted only if:

- Overall F1 improves or stays stable.
- No critical head/class regresses beyond agreed tolerance.
- Count accuracy improves or stays stable.
- Calibration does not get worse.
- Latency remains within acceptable range.
- False negatives for important classes do not increase.

## **6. Human Labels vs Pseudo-Labels**

Human labels and pseudo-labels should be stored separately.

Recommended weighting:

- Human labels: full weight.
- Pseudo-labels: lower weight or filtered only.
- Unverified pseudo-labels: never overwrite human annotations.

## **7. Model Versioning**

Each model version should track:

- Training dataset version.
- Annotation batch IDs.
- Code commit or package version.
- Hyperparameters.
- Validation metrics.
- Calibration metrics.
- Deployment timestamp.
- Rollback path.

# 08 — On-Prem Deployment Plan

## 1. Goal

Deploy the AI inference service inside the infrastructure to avoid network latency and protect proprietary image/research data.

## 2. Proposed Runtime Components

- FastAPI inference server.
- AdaptiveDETR/DINOv3 model wrapper.
- OCR preprocessing module.
- Postprocessor.
- Uncertainty scorer.
- Local model registry.
- Label Studio connector.
- Local logs/metrics.
- Docker container.

## 3. Docker Structure

Suggested services:

services:

inference-api:

image: bacterial-colony-ai:latest

ports: "8000:8000"

environment:

MODEL\_PATH: /models/current

ENABLE\_UNCERTAINTY: "true"

STORE\_IMAGES: "false"

LABEL\_STUDIO\_URL: http://label-studio:8080

volumes:

- ./models:/models
- ./logs:/logs
- ./review\_queue:/review\_queue

deploy:

resources:

reservations:

devices:

- capabilities: [gpu]

label-studio:

image: heartexlabs/label-studio:latest

ports: "8080:8080"

volumes:

- ./label\_studio\_data:/label-studio/data

#### 4. Performance Benchmarking

You have a very high target throughput. Because high-resolution full-image inference with a large DINOv3/AdaptiveDETR model can be computationally heavy, Phase 1 should define benchmarking rather than guarantee final throughput.

Benchmark metrics:

- p50 latency.
- p95 latency.

- p99 latency.
- GPU memory use.
- Max batch size.
- Images/second by image size.
- OCR overhead.
- Postprocessing overhead.
- Uncertainty scoring overhead.

## **5. Scaling Options**

If one GPU is insufficient:

- Batch inference.
- Async request queue.
- Multiple inference workers.
- Multiple GPUs.
- Triton inference server if needed.
- TensorRT/ONNX optimization if compatible.
- Mixed precision inference.
- OCR routing to reduce unnecessary head computation.

## **6. Failure Handling**

The service should handle:

- Invalid image format.
- Oversized image.
- No detected plate.
- OCR failure.
- Model load failure.

- GPU out-of-memory.
- Label Studio unavailable.
- Unsupported head/class.

## 09 – Roadmap (6 weeks, Phase 2, 3, 4)

### Phase 2 — Core Active Learning + Model Improvement Module

The goal of Phase 2 is to build the intelligence layer around the current AdaptiveDETR/DINOv3 model.

This phase does not focus on final production deployment yet. It focuses on:

- parsing current inference outputs
- supporting 9 detection heads
- calculating uncertainty
- selecting samples for review
- preparing pseudo-label candidates
- designing model improvement logic
- preparing retraining-ready datasets

The current inference JSON already contains normalized bounding boxes and probability outputs, so Phase 2 should build directly around that current response structure.

### Week 1 — Phase 2.1 : Current Output Parser + 9-Head Configuration

I will create the foundation of the active learning module so it can understand the current AdaptiveDETR/DINOv3 inference output.

1. Update all configuration to 9 detection heads

YM, EC, TC, XSA, LM, ETB, BC, LS, ALL

2. Confirm class mapping per head

Before implementing the final logic, we need the exact class count and label names for each head.

3. Build inference response parser

Create a parser for the existing JSON output.

The parser should read:

- boxes
- probabilities
- predicted labels
- counts
- selected head / backbone classification output if available
- image-level metadata

4. Normalize internal data format

Convert the raw model output into a clean internal format used by the active learning module.

## 5. Create module structure

```
active_learning_tool/  
    config.py  
    schema.py  
    response_parser.py  
    head_mapping.py  
    utils.py
```

### Deliverables at End of Week 1

- ✓ 9-head configuration file
- ✓ inference JSON parser
- ✓ normalized internal prediction schema
- ✓ head/class mapping structure
- ✓ first working parser test using current JSON

## **Week 2 — Phase 2.2 : Uncertainty Scoring Engine**

I will build the uncertainty engine that decides whether a prediction should be trusted, pseudo-labeled, or sent for human review.

### 1. Implement detection-level uncertainty

Each detected colony should receive an uncertainty score.

#### Metrics

Least confidence:  $1 - \text{max\_probability}$

Margin uncertainty:  $\text{top\_1\_probability} - \text{top\_2\_probability}$

Entropy: uncertainty based on full class probability distribution

### 2. Implement image-level uncertainty

Since one Petri dish can contain many colonies, we need an overall image uncertainty score.

#### Possible scoring methods

- average detection uncertainty
- maximum detection uncertainty
- percentage of uncertain detections
- weighted score based on colony density
- head-specific uncertainty score

### 3. Implement head-level uncertainty

Because the model has 9 detection heads, uncertainty should be tracked per head.

### 4. Add review decision logic

Define rules such as:

If confidence is very high:

    accept prediction or mark as pseudo-label candidate

If confidence is medium:

keep prediction but do not train from it yet

If confidence is low:

send to human review

## 5. Add threshold configuration

Example

uncertainty:

low\_confidence\_threshold: 0.60

review\_uncertainty\_threshold: 0.65

pseudo\_label\_confidence\_threshold: 0.95

min\_box\_confidence: 0.50

## 6. OCR-Aware Pseudo-Label Filtering

Add OCR consistency checks before accepting pseudo-labels.

If confidence > 95% and OCR result agrees with predicted head/class:

allow pseudo-label candidate

If OCR is unclear:

route to ALL or mark as uncertain

If OCR conflicts with model prediction:

do not pseudo-label automatically

send to review queue

### Deliverables at End of Week 2

- ✓ uncertainty.py module
- ✓ per-box uncertainty scoring
- ✓ per-image uncertainty scoring
- ✓ per-head uncertainty summary
- ✓ configurable review thresholds
- ✓ review\_required decision output
- ✓ OCR-aware pseudo-label filter
- ✓ conflict detection between OCR metadata and model prediction
- ✓ ALL fallback handling for unclear OCR

## Week 3 — Phase 2.3 : Hybrid Sampling + Active Learning Batch Selection

I will build the module that selects which images should go to human review first.

This is important because the goal is not only to find uncertain images. The system should also avoid sending many visually similar images to the analysts.

1. Implement Top-K uncertainty selection

Select the most uncertain images from a batch.

Example:

Input: 10,000 unlabeled images

Output: top 200 images for human review

2. Implement hybrid sampling

Combine multiple signals:

- uncertainty score
- diversity score
- detection density
- selected head
- rare class priority
- OCR routing information
- historical failure patterns

3. Add balanced sampling per head

Avoid selecting only one head/domain.

If review batch size is 200 images, select 30 images(YM), 30 images(EC), 30 images(TC), 30 images(XSA), 20 images(LM), 20 images(ETB), 20 images(BC), 10 images(LS), 10 images(ALL).

The exact balance can be adjusted based on dataset distribution.

4. Create active learning batch export

The module should export selected review samples in a format that can later be pushed to Label Studio.

5. Add reporting for active learning rounds

Generate a summary like:

Total images processed: 10,000

Images selected for review: 200

Pseudo-label candidates: 4,500

Low-confidence detections: 720

Most uncertain head: YM

Highest density plate: sample\_023

6. DVC Dataset Versioning Design

Add DVC tracking for active learning rounds and dataset versions.

### Deliverables at End of Week 3

- ✓ query\_strategy.py module
- ✓ Top-K uncertainty selection
- ✓ hybrid sampling strategy
- ✓ balanced per-head review selection
- ✓ review batch export file

- ✓ active learning round summary report
- ✓ DVC repository structure
- ✓ dataset manifest format
- ✓ active learning round tracking format
- ✓ separation between human-reviewed and pseudo-labeled data
- ✓ model-to-dataset lineage design

## Week 4 — Phase 2.4 : Pseudo-Labeling, Calibration Plan + Retraining Dataset Builder

I will prepare the system to reduce manual annotation further by using high-confidence predictions safely and building retraining-ready datasets.

### 1. Implement pseudo-label candidate selection

High-confidence predictions are saved as pseudo-label candidates.

#### Rules

- If confidence  $\geq 0.95$  and uncertainty  $\leq 0.05$ :  
mark as pseudo-label candidate
- If confidence is between 0.60 and 0.95:  
keep for monitoring
- If confidence  $< 0.60$ :  
send to review

Thresholds can be changed after validation.

### 2. Add pseudo-label safety filters

Prevent bad pseudo-labels from entering training.

Filters may include:

- confidence threshold
- box size sanity check
- duplicate box filtering
- head/domain consistency check
- OCR metadata consistency check
- minimum/maximum colony size rules

### 3. Design calibration module

Prepare confidence calibration logic:

- Expected Calibration Error, ECE
- reliability curve data
- temperature scaling plan
- threshold tuning per head

This is important because a model may be overconfident. Calibration helps make confidence scores more trustworthy.

### 4. Build retraining dataset builder

The dataset builder should combine:

- original manually labeled data
- newly reviewed Label Studio corrections
- approved pseudo-label candidates
- metadata about model version and active learning round

Output structure:

```
datasets/
  round_001/
    images/
    annotations/
    pseudo_labels/
    reviewed_labels/
    manifest.json
```

## 5. Create Phase 2 integration handoff

Prepare the module for Phase 3 server integration.

Final Phase 2 package:

```
active_learning_tool/
  config.py
  schema.py
  response_parser.py
  uncertainty.py
  query_strategy.py
  pseudo_label.py
  calibration.py
  dataset_builder.py
  reports.py
  tests/
```

## 6. Weighted Training Dataset Builder

Update the retraining dataset builder to support weighted sample types.

### Deliverables at End of Week 4

- ✓ pseudo\_label.py module
- ✓ pseudo-label candidate export
- ✓ pseudo-label safety filtering
- ✓ calibration plan + ECE calculation structure
- ✓ retraining dataset builder
- ✓ Phase 2 module ready for FastAPI integration
- ✓ unit tests and README
- ✓ human-reviewed dataset export
- ✓ pseudo-labeled dataset export
- ✓ merged training manifest
- ✓ configurable pseudo-label loss weight
- ✓ rule preventing pseudo-labels from overwriting human labels
- ✓ DVC tracking hooks for generated datasets

## Phase 2 Final Deliverables

At the end of the 4-week Phase 2, you can receive:

1. active\_learning\_tool Python package
2. 9-head configuration support
3. current inference JSON parser
4. uncertainty scoring module
5. hybrid sampling module
6. pseudo-labeling module
7. calibration planning module
8. retraining dataset builder
9. review batch export format
10. active learning round report generator
11. unit tests
12. Phase 2 README and usage guide

## Phase 3 — Inference Server + HITL Integration

### Week 5 – Phase 3 : Inference Server + HITL Integration

I will turn the Phase 2 active learning module into a usable API service and connect it with the human review workflow.

1. Build FastAPI inference server  
Endpoints:
  - GET /health
  - GET /model/version
  - POST /predict
  - POST /batch\_predict
  - POST /uncertain\_batch
  - POST /feedback
2. Connect target inference flow  
Client system
  - FastAPI Inference Server
  - Image validation + preprocessing
  - OCR module
  - Petri dish metadata / routing hints
  - AdaptiveDETR / DINOv3 model wrapper
  - Raw model outputs
  - Postprocessor
  - Normalized inference response
  - Uncertainty scorer
  - Review required?
3. Add Label Studio integration
  - export uncertain images to Label Studio
  - pre-fill model predictions

- import lab analyst corrections
- convert corrections to retraining format

#### 4. Dockerize the server

Deliver:

Dockerfile  
docker-compose.yml  
.env.example  
requirements.txt  
README\_server.md

#### 5. FastAPI Resource Guardrails

The FastAPI server must include runtime guardrails for GPU memory.

#### 6. `/uncertain\_batch` Endpoint Requirement

The `/uncertain\_batch` endpoint should process already-generated inference results only.

#### 7. DVC Integration Hooks

The server/retraining workflow should create or update DVC-tracked artifacts when new feedback is imported.

### Phase 3 Deliverables

1. FastAPI inference server
2. active learning module connected to server
3. Label Studio export/import scripts
4. feedback endpoint
5. Docker deployment package
6. server documentation
7. VRAM monitor
8. 10 GB VRAM ceiling enforcement
9. request queue or rejection logic
10. maximum batch size configuration
11. safe batch chunking option
12. clear API error response for oversized requests
13. endpoint accepts existing inference JSON results
14. entropy / margin / confidence filtering
15. sorting and Top-K selection
16. completion target: 10,000 results within 15 minutes
17. clear rejection message if raw images are submitted
18. feedback import creates versioned correction files
19. active learning round output is DVC-trackable
20. retraining dataset builder outputs DVC-compatible directories
21. model registry links model version to DVC dataset version

### Phase 4 — Testing, Deployment + Documentation

#### Week 6 – Phase 4 : Testing, Deployment + Documentation

I will validate the full workflow and prepare handoff.

## 1. End-to-end workflow test

Image upload

- inference result
- uncertainty score
- Label Studio review
- feedback import
- retraining dataset creation

## 2. Performance testing

Measure:

- single-image latency
- batch latency
- GPU memory usage
- CPU/RAM usage
- p95 latency
- p99 latency
- practical throughput

## 3. Validation report

Include:

- active learning batch quality
- uncertainty distribution
- pseudo-label candidate quality
- head-level review counts
- failure cases
- recommended thresholds

## 4. Documentation and handoff

Deliver:

- installation guide
- API reference
- active learning usage guide
- Label Studio workflow guide
- retraining dataset guide
- troubleshooting guide

## 5. Latency Benchmark

Acceptance criteria:

Target:  $\leq 0.2$  seconds per Petri dish

Maximum:  $\leq 0.5$  seconds per Petri dish

The benchmark must include:

- OCR routing
- model inference
- bounding-box calculation

- postprocessing
- uncertainty scoring
- JSON response generation

## 6. `/uncertain\_batch` Benchmark

Acceptance criteria:

10,000 already-generated inference results must be filtered, scored, sorted, and selected within 15 minutes.

The test does not include full image inference.

## 7. VRAM Safety Test

Acceptance criteria:

FastAPI server must not exceed 10 GB VRAM under configured workload.

The test must verify:

- idle VRAM usage
- single request VRAM usage
- batch request VRAM usage
- concurrent request behavior
- oversized request rejection or queueing
- no CUDA OOM crash

## 8. DVC Lineage Test

Acceptance criteria:

A final model version must be traceable to the exact dataset version used for training.

The test must verify:

- human-reviewed labels are tracked
- pseudo-labels are tracked separately
- active learning round outputs are tracked
- training config is tracked
- model artifact is tracked
- validation report is linked to model version

## 9. Pseudo-Label Safety Test

Acceptance criteria:

Pseudo-labels are only accepted when confidence > 95% and no OCR conflict exists.

The test must verify:

- pseudo-labels are stored separately
- pseudo-labels never overwrite human labels
- pseudo-labels use lower training loss weight
- conflicting pseudo-labels are sent to review
- unclear OCR routes to ALL or review logic

## Phase 4 Deliverables

1. tested deployment package
2. validation report

3. performance report
4. documentation
5. handoff/training session